Projet Final : Développement d'une Single Page Application Complète

Objectifs

1. Créer une application React de A à Z en respectant une architecture de fichiers professionnelle.
2. Mettre en place un système de navigation multi-pages avec React Router.
3. Gérer un état global asynchrone complexe avec Redux Toolkit et Redux Thunk.
4. Implémenter des formulaires contrôlés et gérer la remontée d'erreurs.
5. Appliquer des optimisations de rendu sur des composants stratégiques.

Contexte du Projet : OmniMonitor v2.0 (Release Candidate)

Après 3 jours de développement itératif, le commanditaire de l'application **OmniMonitor** vous demande de livrer la version finale (v2.0).

Vous partez d'un projet Vite vierge (ou de la base existante que vous nettoyez). Votre mission est de concevoir l'application complète en autonomie, en respectant le cahier des charges détaillé ci-dessous. Le formateur agira en tant que Lead Developer/Architecte pour vous guider en cas de blocage.

Cahier des Charges

1. Routage et Navigation (React Router)

L'application doit posséder un layout principal (contenant une barre de navigation persistante) et gérer les routes suivantes :

- `/` **(Accueil / Dashboard) :** Affiche un résumé global et la grille des stations météo.
- `/station/:id` **(Détails) :** Affiche la fiche technique complète d'une station sélectionnée.
- `/add` **(Nouvelle Station) :** Affiche un formulaire d'ajout d'une nouvelle station.
- `*` **(Erreur 404) :** Une page de secours si l'URL est introuvable.

2. État Global et Appels Réseau (Redux Toolkit)

Toutes les données des stations doivent être stockées dans Redux.

- Au démarrage de l'application, un `createAsyncThunk` doit simuler un appel réseau pour récupérer la liste initiale des stations (avec un délai de 1 à 2 secondes).

- Gérez visuellement les états d'attente (`loading`) et d'erreur (`failed`) lors de ce chargement.
- Le Store doit permettre d'ajouter une nouvelle station et de supprimer une station existante.

3. Fonctionnalités Attendues par Page

Page Dashboard (`/`)

- Une barre de recherche texte permettant de filtrer instantanément les stations par leur nom.
- Une grille affichant toutes les stations sous forme de cartes (`<StationCard />`).
- Chaque carte doit afficher : le nom, le statut et un bouton "Voir les détails" (lien vers `/station/:id`).
- **Performance :** La grille de cartes doit être optimisée avec `React.memo` et `useCallback` pour éviter de re-rendre toutes les cartes lorsque l'utilisateur tape dans la barre de recherche.

Page Détails (`/station/:id`)

- Récupère l'ID depuis l'URL (`useParams`).
- Lit les informations détaillées de cette station depuis le Store Redux (`useSelector`).
- Affiche toutes les métadonnées (ID, altitude, statut, date de mise en service).
- Fournit un bouton "Supprimer" qui retire la station du Store Redux et redirige automatiquement l'utilisateur vers le Dashboard (`useNavigate`).

Page d'Ajout (`/add`)

- Un formulaire contrôlé comportant : Nom (texte), Altitude (nombre), et Statut initial (select : online/offline).
- Validation : Le bouton de soumission doit être désactivé si le nom contient moins de 3 caractères.
- À la soumission : l'action `preventDefault()` est appelée, une nouvelle station est expédiée dans le Store Redux (`dispatch`), et l'utilisateur est redirigé vers le Dashboard.

Étapes de Développement Recommandées

Ne codez pas tout en même temps ! Suivez cette progression itérative :

1. **Initialisation et Architecture :** Créez vos dossiers `pages`, `components`, `store`, et configurez le Store Redux vide avec son `<Provider>`.
2. **Plomberie du Routage :** Mettez en place `<BrowserRouter>`, `<Routes>`, et vos composants de pages avec du texte factice. Testez la navigation.

3. **Le Cœur (Redux) :** Codez votre `stationsSlice` (état initial vide, thunk pour le mock API, reducers d'ajout et suppression).
4. **Vues et Connexion :** Branchez vos pages au Store (`useSelector`, `useDispatch`) et développez l'interface utilisateur.
5. **Formulaires :** Implémentez la logique de la page d'ajout.
6. **Optimisation :** Lancez le React Profiler, tapez dans la barre de recherche, constatez les ralentissements éventuels et corrigez avec `React.memo` / `useMemo`.

Critères de Validation

- L'application démarre sans erreurs dans la console.
- Le rechargement des pages via la barre de navigation ne provoque pas de requête serveur complète (la navigation est gérée par le client).
- L'ajout, la suppression et le filtrage fonctionnent parfaitement et se répercutent sur toutes les vues.
- Le code est découpé proprement : un fichier par composant, et une séparation claire entre logique métier (Custom Hooks, Redux) et logique d'affichage.